Machine Learning for Customer Churn Prediction

Nelson David Posso Suarez
Javier Alejandro Penagos Hernandez
Nicolas Castro Rivera

1 Data Preparation

In this workshop the churn problem is approached from a different angle. Instead of summarising each subscriber into a single tabular row, as in Workshop 1, we use the dates in the dataset to treat churn prediction as a **sequence problem**: each subscriber is represented by the day-by-day evolution of their listening activity and subscription status. This requires transforming the raw data in a different way from the previous workshop. The pipeline is implemented in `workshops/w2/w2_sequence_preprocessing.ipynb` and the modules under `workshops/w2/src/data_processing/`.

1.1 Structure of the Raw Data

The dataset covers approximately 2015 to 2017 and contains the history of each subscriber's transactions and music listening logs. Three of the raw tables are used to build the sequences: the labelled cohorts (Table 1), the daily listening logs (Table 2) and the payment transactions (Table 3). The member demographics table is not used in this workshop.

Feature	**Description**	**Data type**
msno	Hashed subscriber identifier (primary key)	`str`
is_churn	Churn label: 1 if the user did not renew within 30 days of expiry, 0 otherwise	`int8`

Table 1: Schema of `train.csv` (February cohort) and `train_v2.csv` (March cohort).

Feature	Description	Data type
msno	Hashed subscriber identifier (foreign key)	`str`
date	Listening date in YYYYMMDD format	`int32`
num_25	Number of songs played less than 25% of their duration	`int32`
num_50	Number of songs played between 25% and 50% of their duration	`int32`
num_75	Number of songs played between 50% and 75% of their duration	`int32`
num_985	Number of songs played between 75% and 98.5% of their duration	`int32`
num_100	Number of songs played at least 98.5% of their duration	`int32`
num_unq	Number of unique songs played on that day	`int32`
total_secs	Total seconds of music listened to on that day	`float64`

Table 2: Schema of `user_logs.csv` and `user_logs_v2.csv`.

Feature	Description	Data type
msno	Hashed subscriber identifier (foreign key)	`str`
payment_method_id	Payment method encoded as an integer code	`int8`
payment_plan_days	Number of days in the purchased subscription plan	`int16`
plan_list_price	Listed price of the plan in New Taiwan Dollars (NTD)	`int16`
actual_amount_paid	Amount actually paid by the subscriber in NTD	`int16`
is_auto_renew	Automatic renewal flag (1 = yes, 0 = no)	`int8`
transaction_date	Date of the transaction in YYYYMMDD format	`int32`
membership_expire_date	Membership expiration date after the transaction in YYYYMMDD format	`int32`
is_cancel	Cancellation flag (1 = yes, 0 = no)	`int8`

Table 3: Schema of `transactions.csv` and `transactions_v2.csv`.

1.2 Tensor Construction

1.2.1 Temporal Split and Window Overlap

Each cohort c is defined by a cutoff date τ_c, the last day whose information may be used. Its sequence window contains the $L = 90$ consecutive calendar days that end on the cutoff:

$$W_c = \{\, \tau_c - 89,\ \tau_c - 88,\ \ldots,\ \tau_c \,\}, \qquad |W_c| = L = 90. \tag{1}$$

A calendar date $d \in W_c$ is mapped to the day index

$$t(d) = d - (\tau_c - 89) \in \{0, 1, \ldots, 89\}, \tag{2}$$

so $t = 0$ is the oldest day of the window and $t = 89$ is the cutoff. Table 4 lists the two cohorts.

Cohort	**Role**	**Window W_c**	**Target**	**Users N_c**	**Churn rate**
`train_feb`	Training	2016-11-03 to 2017-01-31	February 2017	992,931	6.4%
`test`	Test	2016-12-01 to 2017-02-28	March 2017	970,960	9.0%

Table 4: Labelled cohorts, their 90-day windows and churn targets.

We build the training tensor from each user's activity between 2016-11-03 and 2017-01-31, with February 2017 churn as the target. The test tensor uses the window from 2016-12-01 to 2017-02-28, with March 2017 churn as the target. The two windows overlap by 62 days (2016-12-01 to 2017-01-31), as shown in Figure 1.

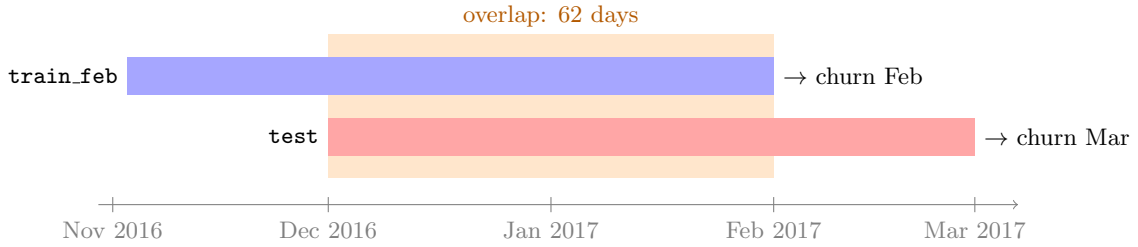


Figure 1: 90-day windows of the two cohorts. Each window ends at its cutoff (2017-01-31 and 2017-02-28) and is followed by its target month.

This overlap is intentional. It follows from using a fixed-length lookback window that advances one month at a time, which is how a subscription service would score its customers in production. Each month, the model receives the most recent 90 days of behaviour and predicts whether the customer will renew in the following month, so consecutive scoring windows necessarily share most of their history.

The overlap does not leak target information. Every feature is computed using only data up to its window's cutoff, and the labels belong to different, non-overlapping periods (February for training, March for testing). During training the model never observes the March outcome, nor any activity after 2017-01-31. The test set therefore measures forward-looking temporal generalisation: how well a model trained on one month's snapshot predicts the next month's churn, including any behavioural drift between the two months.

Many users appear in both cohorts (881,701 of the February users are also in the March cohort), as they would in real monthly scoring. A shared integer identifier `uid` is assigned over the union of both cohorts (1,082,190 unique users) and is used only to join each user with their logs and transactions. Each cohort keeps its own complete list of users, its own row order and its own labels: a user present in only one cohort appears only in that cohort's tensor, and a user present in both appears twice, with a

different cutoff, sequence and label each time. Our results should therefore be read as performance on an existing customer base rather than on entirely unseen users.

1.2.2 Cleaning

Transactions. Both transaction files are concatenated, restricted to the target users and to transactions on or before the cutoff. Exact duplicates are then removed: two rows are considered duplicates when the user (`msno`) and all eight transaction fields (`payment_method_id`, `payment_plan_days`, `plan_list_price`, `actual_amount_paid`, `is_auto_renew`, `transaction_date`, `membership_expire_date`, `is_cancel`) are equal. Only the first copy is kept. Two rows on the same day with different values are treated as different payments and both are kept.

Listening logs. The raw logs may contain several rows for the same user and day. Before aggregation, negative counters are clipped to zero and non-finite values of `total_secs` are replaced by zero. The rows of the same user and day are then summed into a single row. For example, if user A has three rows on 2017-02-01, with `num_25` equal to 3, 2 and 1, the aggregated row has `num_25` $= 6$; the other counters are summed in the same way. Formally, for user i, day d and counter k:

$$v_{i,d,k} = \sum_{r \in R_{i,d}} \max(0,\, v_{r,k}), \tag{3}$$

where $R_{i,d}$ is the set of raw log rows of user i on day d. The number of raw rows $|R_{i,d}|$ is kept to define the activity flag.

1.2.3 Listening Channels

The aggregated logs are placed into the window. Days without any log row are filled with zeros. The seven counters are transformed with $\log(1 + x)$ to compress their heavy-tailed distribution; standard scaling is fitted later, on the training split only, inside the model notebook. An eighth channel, `active`, indicates whether a log row exists for that day, independently of the counter values:

$$x_{i,t,k} = \log(1 + v_{i,d,k})\,, \quad k = 1, \ldots, 7, \qquad a_{i,t} = \mathbf{1}[\,|R_{i,d}| > 0\,]\,, \tag{4}$$

with $t = t(d)$. A day with a log row where every counter is zero is still active, which distinguishes "logged in but did not listen" from "no record". Table 5 lists the listening channels.

Index	Channel	Description	Encoding
0	num_25	Songs played less than 25%	$\log(1+x)$
1	num_50	Songs played 25–50%	$\log(1+x)$
2	num_75	Songs played 50–75%	$\log(1+x)$
3	num_985	Songs played 75–98.5%	$\log(1+x)$
4	num_100	Songs played at least 98.5%	$\log(1+x)$
5	num_unq	Unique songs played	$\log(1+x)$
6	total_secs	Seconds listened	$\log(1+x)$
7	active	1 if a log row exists for the day, 0 otherwise	flag

Table 5: Listening channels (8) of the sequence tensor.

1.2.4 Transaction Channels: Events and States

The tensor is completed with twelve transaction channels, divided into two groups:

- **Events** encode what happened on a specific day. They are written only on the days with at least one payment and are zero on every other day.
- **States** encode the subscription status known at the end of a specific day. They are carried forward to the following days until the next payment updates them.

First, the transactions of each user are aggregated per day. Event values are summed over all payments of the day. State values (cancellation, auto-renewal and expiry date) are taken from the last record of that day in source order, and a flag marks the day as ambiguous when the records of the same day disagree on any of these three values.

Payments before the window. Only the **latest payment day before the window** is kept from the earlier history; all earlier payments are summarised by that single record. This record initialises the state at $t = 0$ but does not create an event inside the window. A payment on day t writes the event channels on day t and updates the state from day t onward. Future payments never modify earlier days, which guarantees that the tensor is causal. This property is verified in the notebook by comparing the vectorised builder against a day-by-day reference implementation and checking that adding a later payment leaves every earlier day unchanged.

Formally, let P_i be the set of payment days of user i up to the cutoff, and let

$$s_i(t) = \max\{\, p \in P_i : p \le d_t \,\} \tag{5}$$

be the most recent payment day known on day d_t (possibly before the window). The state is known

when $s_i(t)$ exists, and the recency and expiry channels are

$$\text{tx_days_since_last}_{i,t} = \log\Big(1 + \text{clip}\left(d_t - s_i(t),\, 0,\, 3650\right)\Big), \tag{6}$$

$$\text{tx_days_to_expire}_{i,t} = \text{slog}\Big(\text{clip}\left(e_i(t) - d_t,\, -3650,\, 3650\right)\Big), \tag{7}$$

where $e_i(t)$ is the membership expiry date recorded on day $s_i(t)$, and $\text{slog}(x) = \text{sign}(x)\log(1+|x|)$ is the signed logarithm, used for quantities that can be negative (an already expired membership, or a negative amount paid). When the state is not yet known, all state channels are zero.

Worked example. Consider a user of the `test` cohort, whose window starts on 2016-12-01 ($t = 0$). The user paid on 2016-11-20, before the window, with auto-renewal disabled and a membership that expires on 2016-12-20, and made no further payments. The pre-window payment initialises the state, so $s_i(t) =$ 2016-11-20 and $e_i(t) =$ 2016-12-20 for every day of the window, and no event is written on any day. Table 6 shows the two channels on three days. On day 19 the membership expires and `tx_days_to_expire` is zero; from day 20 on it is negative, a strong signal that the user has not renewed. These values were obtained by running this user through the same builder used for the export.

Day t	**Date** d_t	$d_t - s_i(t)$	**tx_days_since_last**	$e_i(t) - d_t$	**tx_days_to_expire**
0	2016-12-01	11	$\log(1+11) = 2.48$	19	$\text{slog}(19) = 3.00$
19	2016-12-20	30	$\log(1+30) = 3.43$	0	$\text{slog}(0) = 0.00$
24	2016-12-25	35	$\log(1+35) = 3.58$	-5	$\text{slog}(-5) = -1.79$

Table 6: Recency and expiry channels for a user whose only payment (2016-11-20, expiring 2016-12-20) falls before the `test` window.

Table 7 describes each transaction channel.

Index	**Channel**	**Group**	**Description**	**Encoding**
8	tx_count	Event	Number of payments on the day	$\log(1+x)$
9	tx_paid_sum	Event	Total amount paid on the day (NTD)	$\text{slog}(x)$
10	tx_plan_days_sum	Event	Total plan days purchased on the day	$\log(1+x)$
11	tx_cancel_count	Event	Number of cancellation records on the day	$\log(1+x)$
12	tx_auto_renew_count	Event	Number of auto-renew records on the day	$\log(1+x)$
13	tx_state_known	State	1 once at least one payment has been observed up to the day, including the pre-window payment	flag

Continued on next page

Index	Channel	Group	Description	Encoding
14	tx_last_is_cancel	State	Cancellation flag of the most recent payment day	flag
15	tx_last_is_auto_renew	State	Auto-renew flag of the most recent payment day	flag
16	tx_days_since_last	State	Days since the most recent payment, clipped to $[0, 3650]$	$\log(1+x)$
17	tx_days_to_expire	State	Membership expiry minus the current day, clipped to $[-3650, 3650]$	$\mathrm{slog}(x)$
18	tx_expiry_known	State	1 if the expiry date of the most recent payment is valid	flag
19	tx_state_ambiguous	State	1 if the records of the most recent payment day disagree on cancellation, auto-renewal or expiry	flag

Table 7: Transaction channels (12) of the sequence tensor.

1.2.5 Resulting Tensor

For each user i of cohort c, the 20 channels are stacked into a matrix whose rows are the 90 days, ordered from oldest to newest, and whose columns are the channels in the order of Tables 5 and 7:

$$X_i^{(c)} = \begin{bmatrix} x_{i,0,0} & x_{i,0,1} & \cdots & x_{i,0,19} \\ x_{i,1,0} & x_{i,1,1} & \cdots & x_{i,1,19} \\ \vdots & \vdots & \ddots & \vdots \\ x_{i,89,0} & x_{i,89,1} & \cdots & x_{i,89,19} \end{bmatrix} \in \mathbb{R}^{L \times F}, \qquad L = 90,\ F = 20. \tag{8}$$

Stacking the N_c users of the cohort gives the cohort tensor

$$\mathcal{X}^{(c)} = \left[\, X_1^{(c)}, X_2^{(c)}, \ldots, X_{N_c}^{(c)} \,\right] \in \mathbb{R}^{N_c \times L \times F}, \tag{9}$$

together with the label vector $y^{(c)} \in \{0,1\}^{N_c}$, aligned so that position i of $y^{(c)}$ is the label of $X_i^{(c)}$. The channel dimension splits into three blocks:

$$X_i^{(c)} = \left[\, \underbrace{X_i^{\text{listen}}}_{\mathbb{R}^{90\times 8}} \;\middle|\; \underbrace{X_i^{\text{event}}}_{\mathbb{R}^{90\times 5}} \;\middle|\; \underbrace{X_i^{\text{state}}}_{\mathbb{R}^{90\times 7}} \,\right]. \tag{10}$$

In this way we obtain $\mathcal{X}^{(\text{train_feb})} \in \mathbb{R}^{992{,}931\times 90\times 20}$ and $\mathcal{X}^{(\text{test})} \in \mathbb{R}^{970{,}960\times 90\times 20}$. The tensors are

stored as `float16` arrays (`seq_lstm_train_feb.npy` and `seq_lstm_test.npy`, about 3.5 GB each) with their labels in `labels_<cohort>.parquet`, in the directory `workshops/w2/export-sequential-v5/`. Before export, an audit rejects the tensor if any of the following fails: the expected shape, finite values, binary flags, non-negative counts and recency, zero event channels on days without a payment, and zero state channels before the state is known.

2 Model

We use a recurrent neural network, the Long Short-Term Memory network (LSTM). We choose this architecture because it is designed for ordered sequences: it reads the 90 days one at a time and keeps an internal memory of the customer's past behaviour, which suits the daily tensors built in the previous section. The model is implemented in PyTorch in the class `LSTMModel` (`workshops/w2/src/models/lstm/model.py`). Defining it as a class lets us reuse the same model and change its size (number of layers and hidden units) without modifying the code. The training and evaluation are orchestrated in `workshops/w2/w2_neural_models.ipynb`.

2.1 Architecture

The architecture has the following layers (Figure 2):

- **Input layer.** A batch of users taken from the tensor built before, with shape $(B, 90, 20)$: B users, 90 days and 20 channels. Each day t of user i is the vector $x_{i,t} \in \mathbb{R}^{20}$.
- **Hidden dimension** (`hidden_dim`). The number of units in the hidden state $h_t \in \mathbb{R}^H$ of each LSTM layer. It sets how much information the network can store about the customer.
- **Layer dimension** (`layer_dim`). The number of stacked LSTM layers, between 2 and 4. The first layer reads the 20 channels of each day; each following layer reads the hidden states of the layer below. Inside each cell, the gates use the sigmoid function and the candidate memory and output use tanh, which introduces the nonlinearity and keeps the hidden state bounded in $[-1, 1]$.
- **Batch normalisation layer.** After the 90 days, the output of the last layer on the last day, $h_{90} \in \mathbb{R}^H$, has shape (B, H). This vector summarises the customer's behaviour over the 90 days. `BatchNorm1d` normalises it across the batch, which stabilises the training.
- **Output layer.** A linear layer applies the transformation $z = w^\top \tilde{h}_{90} + b$ and produces one value per user, the *logit* $z \in \mathbb{R}$.

For each day, an LSTM layer updates its memory cell c_t and hidden state h_t with

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \qquad i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \tag{11}$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t), \tag{12}$$

where f_t, i_t and o_t are the forget, input and output gates, σ is the sigmoid function and $\odot$ is the element-wise product. The states start at $h_0 = c_0 = 0$. The model output and the churn probability are

$$z_i = w^\top \mathrm{BN}\left(h_{i,90}\right) + b, \qquad \hat{p}_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}. \tag{13}$$

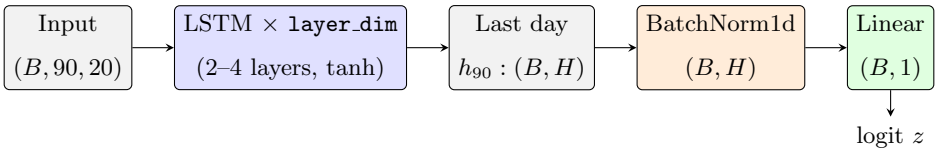


Figure 2: LSTM architecture. B is the batch size and H the hidden dimension. The sigmoid is applied to the logit inside the loss during training and explicitly at prediction time.

2.2 Loss Function and Optimiser

Since churn is a binary classification problem, we use the binary cross-entropy loss. In PyTorch it is defined as `criterion = nn.BCEWithLogitsLoss()`. This function receives the logit z_i and applies the sigmoid internally, so the model does not include a sigmoid layer. Combining both operations in one function is numerically stable: it never evaluates $\log(0)$ when the sigmoid saturates. For a batch of B users the loss is

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \Big[y_i \log \sigma(z_i) + (1 - y_i) \log\left(1 - \sigma(z_i)\right) \Big]. \tag{14}$$

At prediction time, where there is no loss, the probability is obtained explicitly with `torch.sigmoid(logits)`.

Backpropagation. Figure 3 summarises how the network learns. In the **forward pass** (black), each layer transforms the output of the previous one:

- **Input:** the 90 days of the user, each a vector of 20 channels.
- **LSTM:** reads the days in order and updates its hidden state, which accumulates the customer's history up to day 90.
- **BatchNorm:** normalises the final hidden state h_{90} so that its scale stays stable during training.
- **Linear:** combines the normalised hidden state into a single score, the logit z.
- **Sigmoid:** turns the logit into a churn probability $\hat{p}$ between 0 and 1.
- **BCE loss:** compares $\hat{p}$ with the true label y and measures the error.

In the **backward pass** (red), the error travels in the opposite direction, from the loss to the input. Each layer receives how much it contributed to the error and passes it on to the layer before. Inside the LSTM, the error also travels back through the 90 days, from the last day to the first. With this information, AdamW adjusts the weights of every layer to reduce the error in the next batch.

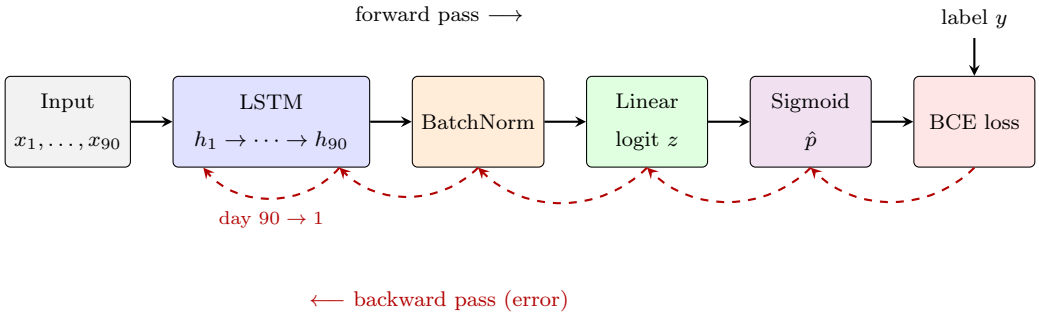


Figure 3: Forward pass (black) and backward pass (red, dashed) for one user. The forward pass turns the 90-day sequence into a probability and a loss; the backward pass sends the error back through every layer and, inside the LSTM, through the 90 days.

The weights are updated with the **AdamW** optimiser, which computes an adaptive step for each weight from the gradient and applies weight decay separately from that step. It receives two parameters:

- **Learning rate** (η): how large each update step is.
- **Weight decay** (λ): shrinks the weights towards zero at every step, which penalises large weights and reduces overfitting.

The function `_optimizer_step` (`workshops/w2/src/models/lstm/train.py`) performs one update. It calls `loss.backward()`, which computes the gradient of the loss with respect to every weight by backpropagation through time; it clips the gradient norm to 1.0, which prevents exploding gradients in the LSTM; and it calls `optimizer.step()` to update the weights. On GPU the forward pass runs in mixed precision (`float16`) to speed up training. The model uses no dropout; overfitting is controlled with weight decay, batch normalisation and early stopping.

2.3 Hyperparameter Search

To find the best hyperparameters we use **Optuna**, a library that searches for the combination that minimises an objective under a given search space. It uses the Tree-structured Parzen Estimator (TPE) sampler, which proposes new configurations based on the results of the previous ones. The search space is defined in `workshops/w2/src/models/lstm/config.py` (Table 8). The objective is the log-loss on the `tune` partition, and the budget is 5 trials.

Hyperparameter	**Search space**
`layer_dim`	Integer in $[2, 4]$
`hidden_dim`	$\{32, 64, 128\}$
Learning rate η	$[10^{-4}, 10^{-2}]$, log scale
Weight decay λ	$[10^{-6}, 10^{-2}]$, log scale
Batch size	Fixed at 512

Table 8: Optuna search space for the LSTM.

3 Training

3.1 Data Split

The February cohort is divided into three disjoint partitions, and the March cohort is kept entirely for the final test (Table 9):

- **Fit** (75%): trains the model.
- **Stop** (12.5%): validates the model after every epoch for early stopping.
- **Tune** (12.5%): evaluates each Optuna configuration and selects the decision threshold.

Each user is assigned to a partition by a stable hash of their `msno`, stratified by label, so the split is reproducible and the three partitions keep the same churn rate. Optuna trains each configuration on `fit`, controls overfitting with `stop` and evaluates it on `tune`. The `test` cohort takes part in no decision.

Partition	Cohort	Users	Churners	Churn rate
`fit`	February	744,699	47,604	6.39%
`stop`	February	124,117	7,934	6.39%
`tune`	February	124,115	7,933	6.39%
`test`	March	970,960	87,330	8.99%

Table 9: Partitions used for training, tuning and testing.

Before training, the numeric channels are clipped at upper quantile limits and standardised with a mean and standard deviation computed only on the days where the channel is defined (active days for listening, payment days for events, days with a known state for states); the binary flags are left unchanged. During the search these parameters are fitted only on `fit`, so no information from `stop`, `tune` or `test` enters the scaling.

3.2 Training Procedure

Each Optuna trial trains an LSTM on `fit` with batches of 512 users for at most 40 epochs. After every epoch the loss is computed on `stop`; if it does not improve for 5 consecutive epochs (*patience*), training stops and the weights of the best epoch are restored. The restored model then predicts `tune`, and its log-loss is the value returned to Optuna:

$$\text{log-loss} = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\Big]. \tag{15}$$

We use log-loss as the main criterion because it evaluates the quality of the predicted probabilities, not only the final decision.

3.3 Optuna Results

Table 10 lists the trials and Figure 4 shows how the objective evolved. Trial 0 failed during execution, so 4 of the 5 trials completed. Trial 1 repeated the configuration of trial 0, because the sampler is seeded, and obtained the best tune log-loss (0.14656). The following trials did not improve on it: the deeper network of trial 2 (4 layers) and the wider network of trial 3 (128 units, about 13 times more parameters) obtained a higher log-loss. Trial 4, close to the best configuration, came within 0.0003. The small differences between configurations (0.1466 to 0.1497) suggest that, for this data, the size of the network matters less than the information in the input channels.

Trial	Layers	Hidden	η	λ	Params	Best ep.	Tune loss	State
0	2	32	2.29×10^{-3}	5.37×10^{-4}	–	–	–	Failed
1	2	32	2.29×10^{-3}	5.37×10^{-4}	15,457	10	**0.14656**	Complete
2	4	32	5.96×10^{-4}	1.45×10^{-5}	32,353	6	0.14971	Complete
3	2	128	7.63×10^{-3}	8.23×10^{-3}	209,281	5	0.14818	Complete
4	2	32	3.49×10^{-3}	1.17×10^{-3}	15,457	11	0.14682	Complete

Table 10: Optuna trials. *Best ep.* is the epoch restored by early stopping and *Tune loss* the log-loss on `tune`.

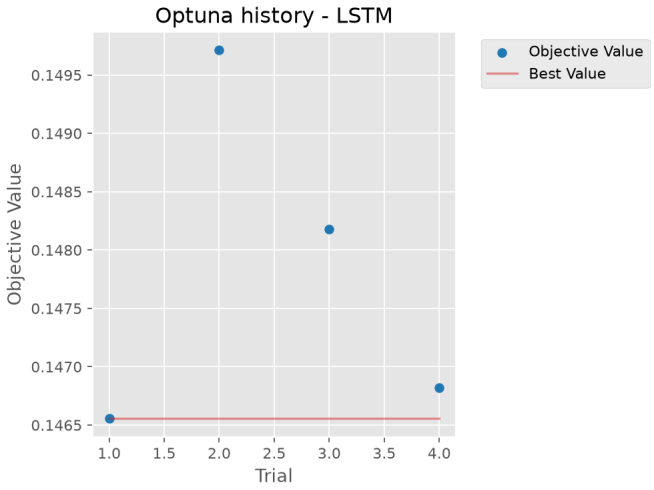


Figure 4: Optuna history: tune log-loss of each completed trial and best value found.

The selected configuration is a 2-layer LSTM with 32 hidden units (15,457 parameters), $\eta = 2.29 \times 10^{-3}$ and $\lambda = 5.37 \times 10^{-4}$, whose best epoch was the 10th.

Decision threshold. The model outputs a probability, and a threshold converts it into a churn / no-churn decision. Because churners are only 6.4% of February, the default threshold of 0.5 classifies very few users as churners. The threshold is therefore selected on `tune` as the value in $\{0.01, 0.02, \ldots, 0.99\}$ that maximises the F1-score of the churn class. The selected threshold is 0.28, and it is frozen before the test.

Final model. With the selected hyperparameters, the model is retrained on the whole February cohort (`fit` + `stop` + `tune`, 992,931 users) for the 10 epochs found during the search, with the scaling parameters refitted on all of February. This model predicts March once.

4 Test

4.1 Metrics

The final model is evaluated once on the March cohort (970,960 users, 8.99% churners). We report two groups of metrics. Probability and ranking metrics do not depend on the threshold: **log-loss** and **Brier score** (mean squared error of the probability) measure the quality of the probabilities, and **ROC-AUC** and **average precision** (the area under the precision-recall curve) measure how well churners are ranked above non-churners. Decision metrics are computed at the threshold: **precision** (of the users flagged as churners, how many churned), **recall** (of the churners, how many were flagged), **F1-score** of the churn class, macro F1, balanced accuracy and the Matthews correlation coefficient (**MCC**), which accounts for all four cells of the confusion matrix and is informative under class imbalance. Accuracy is reported for completeness, but with 91% non-churners a model that never predicts churn would already reach 0.91.

Table 11 reports the results at the threshold selected on `tune` (0.28) and, for reference, at 0.5.

Metric	**Threshold 0.28 (tune)**	**Threshold 0.5**
Log-loss	0.2076	
Brier score	0.0551	
ROC-AUC	0.8507	
Average precision	0.5801	
Accuracy	0.9319	0.9271
Balanced accuracy	0.7455	0.6292
Precision (churn)	0.6534	0.7780
Recall (churn)	0.5181	0.2658
Specificity	0.9728	0.9925
F1 (churn)	0.5780	0.3962
F1 (macro)	0.7705	0.6787
MCC	0.5458	0.4282
Predicted churn rate	7.13%	3.07%

Table 11: LSTM test metrics on the March cohort.

The threshold selected in February transfers well to March: compared with 0.5, it almost doubles the recall (from 0.27 to 0.52) at a moderate loss of precision (from 0.78 to 0.65), and raises the churn F1 from 0.40 to 0.58 and the MCC from 0.43 to 0.55.

4.2 Confusion Matrix

Figure 5 shows the confusion matrix at the 0.28 threshold. The model identifies 45,247 of the 87,330 churners (51.8%) and misses 42,083 (false negatives). It flags 23,998 of the 883,630 loyal customers as at risk (2.7%, false positives). In other words, about two out of three users flagged by the model actually churn.

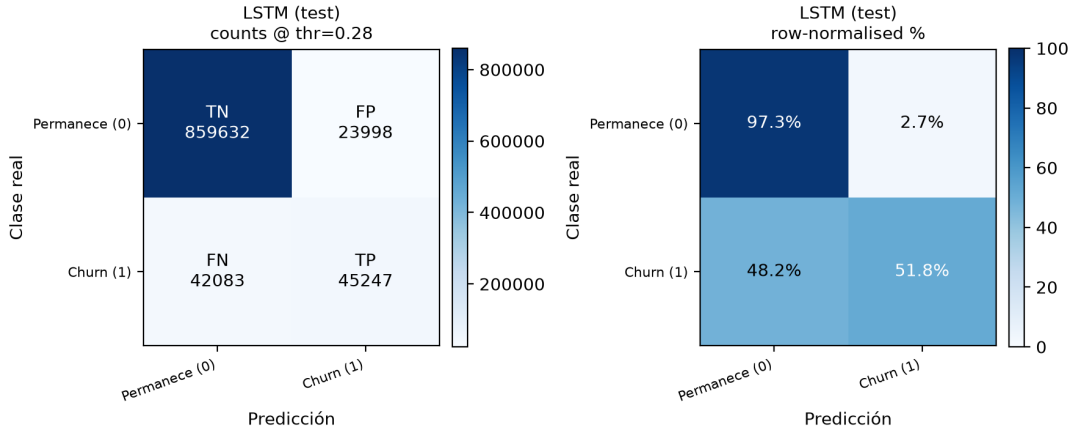


Figure 5: Confusion matrix on the March test cohort at the 0.28 threshold: counts (left) and percentages per true class (right).

4.3 ROC and Precision-Recall Curves

Figure 6 shows the ROC and precision-recall curves on the test cohort. The ROC curve rises steeply at the start: the model reaches a true positive rate of about 0.7 at a false positive rate of about 0.1, so the users with the highest scores are mostly churners. The precision-recall curve keeps a precision close to 1 for the first 10% of recall and stays above 0.6 up to a recall of about 0.55, well above the 9% prevalence line that a random model would obtain. The drop beyond that point corresponds to churners whose 90-day behaviour looks like that of loyal customers.

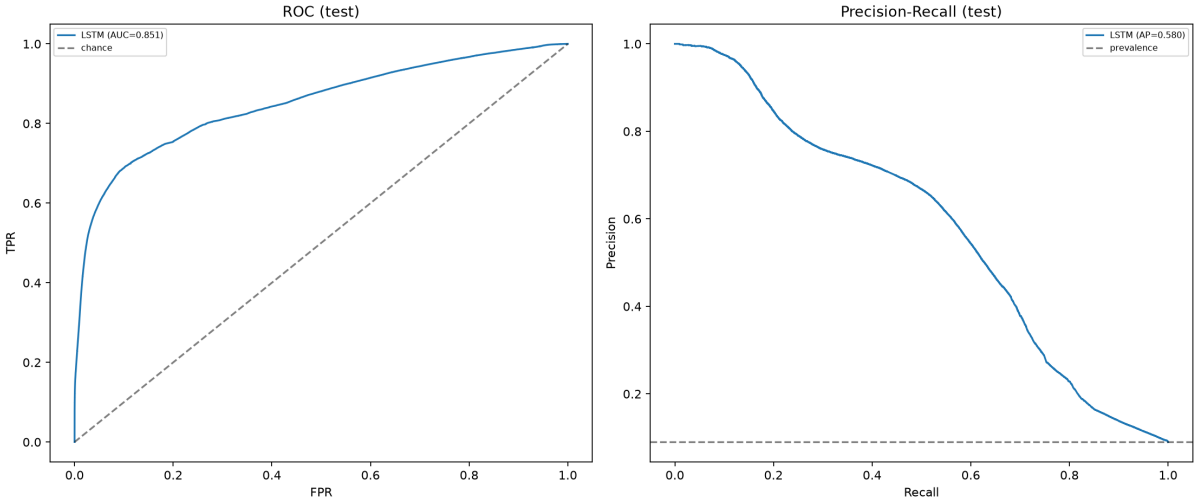


Figure 6: ROC curve (left, AUC = 0.851) and precision-recall curve (right, AP = 0.580) on the March test cohort.

4.4 Calibration

Figure 7 shows the reliability diagram and the distribution of the predicted probabilities. The reliability curve lies above the diagonal: the observed churn rate is higher than the predicted probability (for example, users with a mean predicted probability of 0.45 churn at a rate of about 0.54). The model therefore underestimates churn in March. This is consistent with the change in prevalence between the two months: the model was trained on February, with 6.4% churners, and March has 9.0%. No calibration was applied, so this shift is reported as is.

The histogram shows that most loyal customers receive a probability close to 0, while the churners split into two groups: one spread between 0.1 and 1.0, which the model detects, and one concentrated near 0, which corresponds mostly to the false negatives of the confusion matrix.

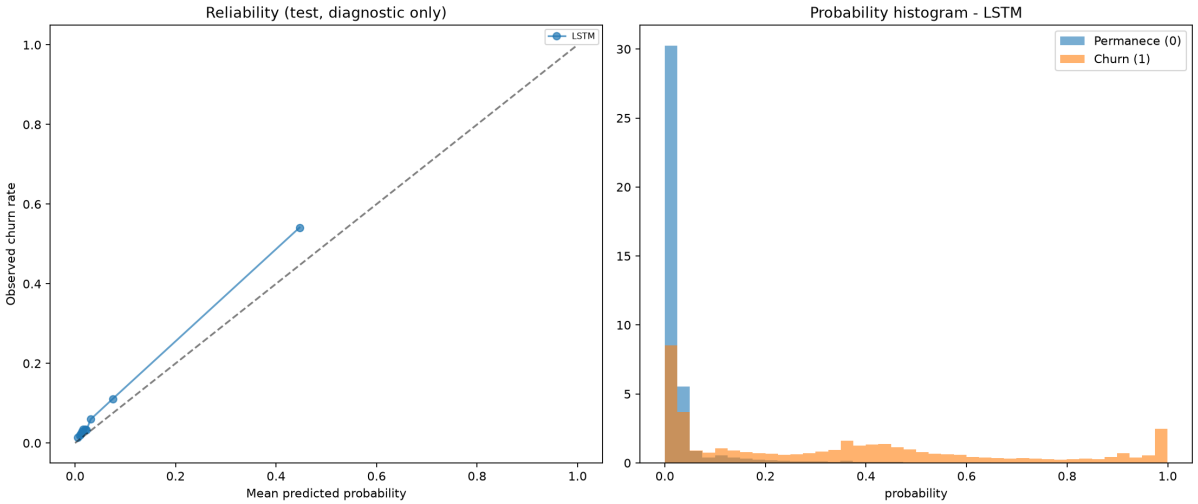


Figure 7: Reliability diagram (left) and histogram of predicted probabilities by true class (right) on the March test cohort.

4.5 Comparison with Workshop 1

Table 12 compares the LSTM with the three tabular models of Workshop 1: logistic regression, random forest and XGBoost. All models are evaluated on the same March cohort, after selecting their thresholds on February, so the comparison is direct. The tabular models use 59 features, including member demographics and the complete transaction history; the LSTM uses only the 90-day sequences of listening and transactions, without demographics.

Model	**Thr.**	**Log-loss**	**Brier**	**AUC**	**AP**	**Prec.**	**Recall**	**F1**	**MCC**
Logistic reg. (W1)	0.28	0.2126	0.0564	0.8445	0.5553	0.5811	0.5343	0.5567	0.5155
Random forest (W1)	0.26	**0.2032**	**0.0548**	**0.8632**	**0.5933**	0.6379	0.5385	**0.5840**	**0.5490**
XGBoost (W1)	0.88	0.4886	0.1498	0.8520	0.5882	0.6094	**0.5424**	0.5740	0.5356
LSTM (W2)	0.28	0.2076	0.0551	0.8507	0.5801	**0.6534**	0.5181	0.5780	0.5458

Table 12: Test metrics on the March cohort for the Workshop 1 tabular models and the Workshop 2 LSTM. Each model uses the threshold selected on its own tuning partition. Best value per column in bold.

The random forest remains the best model on almost every metric, but the margins are small: 0.0044 in log-loss, 0.012 in ROC-AUC, 0.006 in F1 and 0.003 in MCC. The LSTM outperforms logistic regression on every metric, and it obtains the highest precision of all models (0.653): when it flags a user as a churner, it is right more often than any tabular model, at the cost of a slightly lower recall. Against XGBoost, the LSTM is close in ROC-AUC (0.851 against 0.852) and better in F1 and MCC; it also produces much better probabilities (log-loss 0.208 against 0.489), since the XGBoost probabilities of Workshop 1 are poorly calibrated, as its 0.88 threshold reflects.

5 Conclusions

Relation with Workshop 1. Workshop 1 and this workshop solve the same problem on the same data, cohorts and test month, but with two different representations. Workshop 1 summarises each subscriber into one row of 59 hand-crafted features: monthly listening aggregates, month-over-month trends, recency, transaction statistics over the complete history and member demographics. Here each subscriber is a 90×20 sequence of raw daily values, and the LSTM has to learn by itself which patterns matter. Both pipelines follow the same rules: features use only information up to each cohort cutoff, the threshold is chosen on February, and March is predicted once. This makes the results of Table 12 directly comparable. The LSTM reaches a performance close to the random forest (ROC-AUC 0.851 against 0.863, churn F1 0.578 against 0.584), outperforms logistic regression on every metric, and obtains the highest churn precision of all models (0.653).

How the sequence captures behaviour. The LSTM reaches this performance with less information than the tabular models: only the last 90 days, without demographics and without the payment history

before the window, which enters only through the state of the last pre-window payment. The daily tensor keeps the order and timing of events that the tabular features compress into monthly sums: a user who stops listening during the last two weeks, a cancellation followed by inactivity, or a membership that approaches its expiry date without a renewal. The state channels make the subscription status visible on every day, and `tx_days_to_expire` turns negative when a membership expires without a new payment, which is one of the clearest churn signals. That the model reaches this performance from the raw daily sequence, without feature engineering, is the main result of this workshop; we did not analyse which of these patterns it relies on most, which would require an interpretability study. At the same time, the gap with the random forest suggests that the long-term information outside the window (tenure, full payment history and demographics) still carries predictive value that the 90-day sequence does not contain.

Why no further experiments are possible. The main limitation is the dataset itself. The competition publishes official churn labels for only two months: February 2017 (`train.csv`) and March 2017 (`train_v2.csv`). A user is labelled as churn when their membership expires in the target month and they do not subscribe again within the following 30 days, so labelling a month requires the transactions of the next 30 days. Since the published transactions and logs end on 2017-03-31, no later month (April 2017 onwards) can be labelled. Earlier months, such as January 2017 or December 2016, could in principle be labelled by applying the official labelling rule to the transactions, and the listening history before the window is available since 2015. However, our reconstruction of that rule, a port of the official `WSDMChurnLabeller.scala`, agrees with the official labels for only 98.8% of the February users and 97.0% of the March users, and about 137,000 official users per month cannot be reconstructed from the published transactions. Labels for other months would therefore be noisier than the official ones and would not define the same cohorts, so a model trained or tested on them would not be comparable with the results reported here. For this reason the evaluation is limited to one training month and one test month, and we cannot measure how stable the model is across several months.